🏛 GLA UNIVERSITY, MATHURA — DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

HPC Masterclass | Day 3 Post-Assessment: Advanced Deep Learning Systems, oneDNN, Slurm & Distributed Horovod

Target Audience: 7th Semester B.Tech CSE (HPC Elective) | **Total Questions**: 50 | **Duration**: 60 Minutes

📋 Answer Key & Forensic Solution Guide (Instructor Copy)

Q#	Correct Answer	Cognitive Bloom Level	Core Concept Focus
01	B	`Understand`	In Intel oneDNN execution pipelines, what is the a...
02	B	`Analyze`	When oneDNN fuses a Conv2D operation with an eleme...
03	B	`Application`	What information does setting the environment vari...
04	B	`Analyze`	Why does oneDNN introduce 'blocked memory layouts'...
05	B	`Apply`	What is 'weight pre-packing' (weight reordering) i...
06	B	`Understand`	In Intel AMX (Advanced Matrix Extensions), how do ...
07	A	`Apply`	What does oneDNN's 'scratchpad memory' management ...
08	B	`Knowledge`	What low-level hardware instruction set is utilize...
09	B	`Analyze`	When configuring thread affinity for oneDNN on a d...
10	B	`Understand`	Why does oneDNN prefer executing JIT-compiled kern...
11	B	`Understand`	In the Intel Xe-HPC architecture (Intel Data Cente...
12	B	`Knowledge`	What is 'Xe Link' in multi-GPU Intel Data Center G...
13	B	`Understand`	How does Intel Extension for TensorFlow (ITEX) int...
14	B	`Knowledge`	What are the two distinct types of compute cores i...
15	B	`Analyze`	How does Habana Gaudi's integration of 24 on-die 1...
16	B	`Knowledge`	What is the function of the Habana Communication L...
17	B	`Application`	Which Python library and module must be loaded in ...
18	B	`Analyze`	What is 'Lazy Mode' execution in the Habana Synaps...
19	B	`Knowledge`	What command-line utility provides real-time hardw...
20	B	`Understand`	In an 8-card Habana Gaudi server, how are the 24 R...
21	A	`Application`	In an enterprise Slurm script for distributed deep...

Q#	Correct Answer	Cognitive Bloom Level	Core Concept Focus
22	B	`Analyze`	What is the consequence of failing to set '#SBATCH...
23	B	`Application`	What information does the Slurm command 'sacct -j ...
24	B	`Understand`	When a Slurm job fails with exit code 137 (SIGKILL...
25	B	`Analyze`	In a distributed Slurm environment, why is it reco...
26	B	`Knowledge`	What does the Slurm directive '#SBATCH --gres=gpu:...
27	A	`Understand`	How does Slurm enforce CPU core pinning and memory...
28	B	`Understand`	What does a pending job status with 'Reason: Prior...
29	B	`Analyze`	Why is an HPC cluster administrator's 'Prolog' and...
30	B	`Application`	In a Slurm heterogeneous job or array job, what do...
31	B	`Understand`	In the Ring-AllReduce algorithm, how many logical ...
32	B	`Analyze`	Why is the communication time in Ring-AllReduce de...
33	B	`Apply`	In Horovod, why must dataset sharding be implement...
34	B	`Understand`	In a distributed training script using Horovod wit...
35	B	`Analyze`	What is 'Tensor Fusion' in Horovod, controlled by ...
36	B	`Analyze`	What occurs if a developer forgets to call 'hvd.br...
37	B	`Understand`	How does Horovod's 'Auto-Tuning' feature (`--autot...
38	B	`Understand`	When scaling from 1 worker to 64 workers with Horo...
39	B	`Knowledge`	In Horovod, what is the role of the 'Response Cach...
40	A	`Application`	Which command correctly launches a Horovod trainin...
41	B	`Analyze`	What is the primary difference between 'Weak Scali...
42	B	`Analyze`	In distributed deep learning, why does Strong Scal...
43	B	`Understand`	How does 'Communication-Computation Overlap' in mo...
44	B	`Analyze`	If an HPC cluster monitoring profile indicates tha...
45	B	`Understand`	What is the purpose of 'Gradient Clipping by Globa...
46	B	`Understand`	In high-performance RoCE v2 networks, why is Prior...
47	B	`Analyze`	What does a sudden loss value of 'NaN' (Not-a-Numb...
48	B	`Analyze`	Why is storing deep learning datasets in container...
49	B	`Understand`	How does 'Pipeline Parallelism' (e.g., GPipe) over...
50	B	`Knowledge`	In enterprise HPC clusters, what is the role of In...

🔬 Complete Questions with Detailed Technical Rationale

Question 01

In Intel oneDNN execution pipelines, what is the architectural distinction between a 'primitive descriptor' and an executable 'primitive'?

- **(a)** A primitive descriptor stores weights on NVMe disk, while a primitive stores weights in RAM.
- **(b) [CORRECT]** A primitive descriptor defines the mathematical operation, shapes, data types, and engine attributes to query optimal algorithms; a primitive is the instantiated executable object created from the descriptor.
- **(c)** A primitive descriptor only exists in Python, while a primitive only exists in Fortran.
- **(d)** A primitive descriptor compiles machine code, while a primitive decompiles it.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Understand`
> **Architectural Rationale**: In oneDNN, a primitive descriptor encapsulates the operation specification (shapes, data types, post-ops, memory descriptors, target engine). Querying it allows the library to select the most performant JIT kernel before instantiating the executable primitive object.

Question 02

When oneDNN fuses a Conv2D operation with an elementwise ReLU activation and a residual Addition (post-op sum), what specific memory traffic is eliminated?

- **(a)** The read traffic for model weights from L1 cache.
- **(b) [CORRECT]** Two intermediate DRAM write operations (Conv2D output, Sum output) and two intermediate DRAM read operations (reading back for Addition and ReLU).
- **(c)** The network transmission of batch labels over InfiniBand.
- **(d)** The PCIe transfer of dataset files from host to disk.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Analyze`
> **Architectural Rationale**: Without fusion: Conv2D writes tensor to DRAM (1 write), Residual Add reads from DRAM and writes to DRAM (1 read, 1 write), ReLU reads and writes to DRAM (1 read, 1 write). Fusion maintains the tensor in per-core registers/L1 cache across all three steps, eliminating 2 writes and 2 reads to DRAM.

Question 03

What information does setting the environment variable 'ONEDNN_VERBOSE=1' output to the console during neural network execution?

- **(a)** The entire Python abstract syntax tree (AST).
- **(b) [CORRECT]** A detailed execution log for every oneDNN primitive call, including engine type, primitive kind, implementation name (e.g., jit:avx512_core), memory formats, input/output tensor shapes, and kernel execution time in milliseconds.
- **(c)** The CPU temperature and fan RPM speeds.
- **(d)** The operating system user password hashes.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Application`
> **Architectural Rationale**: ONEDNN_VERBOSE=1 prints execution telemetry for every JIT-compiled primitive: timestamp, engine (cpu/gpu), primitive name, JIT kernel implementation (e.g., jit:avx512_core_amx_bf16_convolution), shapes, and wall-clock execution time.

Question 04

Why does oneDNN introduce 'blocked memory layouts' such as 'nChw16c' instead of standard contiguous NCHW or NHWC on AVX-512 enabled Intel Xeon processors?

- **(a)** To encrypt tensor activations to meet security compliance.
- **(b) [CORRECT]** To chunk channels into blocks of 16 contiguous elements matching 512-bit vector register capacity (16 floats x 32 bits = 512 bits), enabling full vector FMA utilization without strided memory gathering.
- **(c)** To convert 4-dimensional tensors into 2-dimensional CSV spreadsheets.
- **(d)** To ensure compatibility with legacy 8-bit microprocessors.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Analyze`
> **Architectural Rationale**: Blocked formats (like nChw16c) pack 16 spatial channel elements consecutively in memory. A single 512-bit vector load instruction (`_mm512_load_ps`) loads all 16 values directly into a ZMM register in 1 cycle without expensive strided gather operations.

Question 05

What is 'weight pre-packing' (weight reordering) in oneDNN inference pipelines, and why is it beneficial for deployment?

- **(a)** Compressing weight files with gzip before uploading to the server.
- **(b) [CORRECT]** Transforming static model weights offline from standard user layout (e.g., OHWI) into the hardware-optimized blocked format (e.g., OIhw16i16o) once at load time, avoiding per-inference layout reordering overhead.
- **(c)** Converting all model weights into random numbers to test stability.
- **(d)** Moving weights from RAM into external USB storage.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Apply`
> **Architectural Rationale**: Because model weights are constant during inference, reordering them into oneDNN's blocked internal layout once during model loading eliminates continuous on-the-fly layout conversion overhead during real-time inference requests.

Question 06

In Intel AMX (Advanced Matrix Extensions), how do the 8 hardware 2D tile registers (TMM0 to TMM7) organize data for the TMUL (Tile Matrix Multiply) unit?

- **(a)** Each tile holds a 1D scalar variable of 64 bits.
- **(b) [CORRECT]** Each tile holds a 2D matrix chunk of up to 16 rows and 64 bytes per row (up to 1024 bytes per tile), allowing the TMUL systolic unit to perform dense matrix multiply-accumulate operations directly from registers.

- **(c)** Each tile stores an entire operating system process table.
- **(d)** Tiles can only store 32-bit integer counters.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Understand`
> **Architectural Rationale**: Intel AMX tile registers provide a 2D register file (1KB per tile, 8KB total across 8 tiles). The TMUL unit reads two tile registers (A and B), performs matrix multiplication, and accumulates into a third tile (C) in high-throughput BF16 or INT8 formats.

Question 07

What does oneDNN's 'scratchpad memory' management allow an HPC developer or framework integrator to optimize?

- **(a) [CORRECT]** It allows allocating temporary working memory buffers from a custom user memory pool rather than having the library call dynamic 'malloc/free' inside high-frequency execution loops.
- **(b)** It provides a text editor inside the Linux terminal.
- **(c)** It formats the swap partition on the boot drive.
- **(d)** It writes log messages to systemd journal.

> **Correct Answer**: **(A)**
> **Bloom's Cognitive Level**: `Apply`
> **Architectural Rationale**: oneDNN primitives require temporary scratchpad memory for intermediate operations. By setting `scratchpad_mode::user`, applications can supply pre-allocated memory pools, completely avoiding dynamic heap allocations on the hot inference path.

Question 08

What low-level hardware instruction set is utilized by oneDNN on 2nd and 3rd Gen Intel Xeon Scalable processors (Cascade Lake / Ice Lake) to achieve $4\times$ throughput on INT8 quantized neural networks?

- **(a)** Intel MMX
- **(b) [CORRECT]** AVX-512 VNNI (Vector Neural Network Instructions) using `VPDPBUSD`
- **(c)** ARM NEON
- **(d)** RISC-V Vector Extension

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Knowledge`
> **Architectural Rationale**: AVX-512 VNNI introduces the `VPDPBUSD` instruction, which multiplies four pairs of 8-bit integers and accumulates the results into a 32-bit integer in a single instruction, replacing 3 separate instructions and quadrupling INT8 throughput.

Question 09

When configuring thread affinity for oneDNN on a dual-socket Intel Xeon system with 64 physical cores, which OpenMP configuration prevents thread migration and false sharing?

- **(a)** OMP_NUM_THREADS=1000; OMP_PLACES=threads

- **(b) [CORRECT]** OMP_NUM_THREADS=64; OMP_PLACES=cores; OMP_PROC_BIND=close
- **(c)** OMP_DYNAMIC=TRUE; OMP_NESTED=TRUE
- **(d)** OMP_WAIT_POLICY=PASSIVE; OMP_NUM_THREADS=1

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Analyze`
> **Architectural Rationale**: Setting `OMP_PLACES=cores` maps threads to distinct physical CPU cores. `OMP_PROC_BIND=close` keeps threads on adjacent cores sharing L3 cache, avoiding inter-socket UPI traffic and NUMA penalties.

Question 10

Why does oneDNN prefer executing JIT-compiled kernels (Just-In-Time) over pre-compiled static library functions for deep learning primitives?

- **(a)** Static libraries cannot run on Linux servers.
- **(b) [CORRECT]** JIT compilation inspects the exact tensor dimensions, strides, padding, and specific CPU microarchitecture at runtime, emitting assembly instructions with hardcoded loop bounds and fully unrolled vector registers.
- **(c)** JIT compilation eliminates the need for physical CPU caches.
- **(d)** JIT kernels do not require any memory allocation.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Understand`
> **Architectural Rationale**: oneDNN's runtime JIT engine generates specialized assembly at runtime for the exact matrix dimensions and padding, baking immediate constants and loop counts directly into machine code for zero branch misprediction.

Question 11

In the Intel Xe-HPC architecture (Intel Data Center GPU Max), what is the function of the Xe Matrix Extension (XMX) systolic engines within each Xe Core?

- **(a)** Handling PCIe root complex interrupts.
- **(b) [CORRECT]** Executing dense matrix multiply-accumulate operations (DPAS - Dot Product Accumulate Systolic) at high arithmetic throughput for FP16, BF16, and INT8 tensors.
- **(c)** Decompressing video streams for web browsers.
- **(d)** Managing hard drive RAID controllers.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Understand`
> **Architectural Rationale**: Each Xe-HPC core contains Vector Engines (XVE) for SIMD operations and XMX systolic matrix engines that compute high-throughput matrix dot-products (DPAS) for deep learning acceleration.

Question 12

What is 'Xe Link' in multi-GPU Intel Data Center GPU Max server deployments?

- **(a)** A web browser extension for monitoring GPU clock speeds.
- **(b) [CORRECT]** A high-speed, direct inter-GPU interconnect fabric that enables coherent memory access and high-bandwidth peer-to-peer communication between GPU tiles without traversing the host PCIe bus.
- **(c)** A serial RS-232 cable for debugging kernel panics.
- **(d)** A software emulator for running CUDA code on AMD GPUs.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Knowledge`
> **Architectural Rationale**: Xe Link provides high-speed bi-directional interconnects between Xe-HPC GPUs (up to 128 GB/s bi-directional per link), allowing all-to-all GPU communication over high-speed traces without PCIe bottlenecks.

Question 13

How does Intel Extension for TensorFlow (ITEX) interface with Intel GPUs under the hood?

- **(a)** By translating TensorFlow graphs into Java bytecode.
- **(b) [CORRECT]** Via standard oneAPI Level Zero driver interfaces and SYCL runtime kernels registered through the TensorFlow PluggableDevice C API.
- **(c)** By running an embedded virtual machine inside the GPU firmware.
- **(d)** Via legacy OpenGL 1.1 graphics pipelines.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Understand`
> **Architectural Rationale**: ITEX leverages the TensorFlow PluggableDevice API to register custom Intel XPU kernels compiled via oneAPI DPC++/SYCL, interacting with hardware via low-level Level Zero APIs.

Question 14

What are the two distinct types of compute cores inside the Habana Gaudi processor architecture?

- **(a)** ALU and FPU
- **(b) [CORRECT]** Matrix Multiplication Engine (MME) and Tensor Processor Cores (TPC)
- **(c)** CPU core and GPU shader core
- **(d)** Cortex-A53 and Cortex-A72

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Knowledge`
> **Architectural Rationale**: Habana Gaudi decouples matrix computations from general-purpose tensor manipulations: the dedicated MME computes heavy GEMMs, while multiple programmable VLIW SIMD TPCs handle activations, normalizations, and custom kernels.

Question 15

How does Habana Gaudi's integration of 24 on-die 100GbE RoCE ports per processor revolutionize multi-node distributed training clusters?

- **(a)** It allows connecting 24 computer monitors directly to each processor.

- **(b) [CORRECT]** It enables direct inter-processor scale-out communication across standard Ethernet switches without needing expensive proprietary NICs (such as PCIe InfiniBand adapters).
- **(c)** It forces all network packets to travel over Wi-Fi.
- **(d)** It limits cluster size to a maximum of 2 nodes.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Analyze`
> **Architectural Rationale**: Each Gaudi 2 processor integrates 24x 100GbE RoCE v2 engines directly on silicon. 21 ports connect all-to-all internally across the 8-card server baseboard, and 3 ports per card route externally for multi-node scale-out over standard COTS Ethernet switches.

Question 16

What is the function of the Habana Communication Library (HCL) within the SynapseAI software stack?

- **(a)** Displaying chat messages between developers.
- **(b) [CORRECT]** Providing high-performance collective communication primitives (AllReduce, AllGather, Broadcast) optimized to exploit Gaudi's on-chip 24x 100GbE RoCE fabric directly.
- **(c)** Translating Spanish documentation into English.
- **(d)** Managing user email notifications for failed Slurm jobs.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Knowledge`
> **Architectural Rationale**: HCL (Habana Communication Library) provides optimized collective operations (AllReduce, ReduceScatter, AllGather) that directly program the on-chip RoCE network engines for zero-copy inter-accelerator communication.

Question 17

Which Python library and module must be loaded in PyTorch or TensorFlow scripts to enable execution on Habana Gaudi accelerators?

- **(a)** import torch_cuda
- **(b) [CORRECT]** import habana_frameworks.torch.core as htcore
- **(c)** import gaudi_hardware_driver
- **(d)** import tensorflow_directx

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Application`
> **Architectural Rationale**: For PyTorch, importing `habana_frameworks.torch.core as htcore` initializes the Gaudi backend (`torch.device('hpu')`), registering custom Gaudi kernels and the SynapseAI lazy evaluation runtime.

Question 18

What is 'Lazy Mode' execution in the Habana SynapseAI PyTorch/TensorFlow bridge, and why is it preferred over Eager Mode on Gaudi processors?

- **(a)** Lazy Mode slows down the processor clock to save electricity during training.

- **(b) [CORRECT]** Lazy Mode accumulates operations into a computational graph before triggering execution, allowing the SynapseAI graph compiler to perform operator fusion, memory planning, and MME/TPC pipeline parallelization.
- **(c)** Lazy Mode executes only odd-numbered layers of the neural network.
- **(d)** Lazy Mode stores weights in temporary text files.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Analyze`
> **Architectural Rationale**: In Lazy Mode, SynapseAI traces operations into a complete graph, allowing the compiler to perform global memory allocation, fuse activation/normalization kernels, and optimize DMA tensor transfers between HBM and MME/TPC units.

Question 19

What command-line utility provides real-time hardware telemetry (utilization, HBM memory usage, power, temperature) for Habana Gaudi processors on Linux?

- **(a)** nvidia-smi
- **(b) [CORRECT]** hl-smi
- **(c)** intel-gpu-tools
- **(d)** rocm-smi

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Knowledge`
> **Architectural Rationale**: `hl-smi` (Habana Labs System Management Interface) is the official telemetry tool for inspecting Gaudi status, HBM memory, power draw, temperature, and RoCE port connectivity.

Question 20

In an 8-card Habana Gaudi server, how are the 24 RoCE 100GbE ports per OAM card typically partitioned between intra-node and inter-node fabrics?

- **(a)** All 24 ports are connected to an external home router.
- **(b) [CORRECT]** 21 ports per card are wired in an all-to-all non-blocking mesh across the 8 internal cards; 3 ports per card (24 ports total per server = 2.4 Tbps) route to external cluster switches.
- **(c)** 1 port is used for data; 23 ports are left disconnected for cooling airflow.
- **(d)** Ports are dynamically reassigned as audio channels.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Understand`
> **Architectural Rationale**: The Gaudi 2 baseboard uses 21 ports per chip for fully connected intra-chassis all-to-all communication among the 8 Gaudi chips, while the remaining 3 ports from each of the 8 chips (24 ports total) provide 2.4 Tbps of external scale-out networking.

Question 21

In an enterprise Slurm script for distributed deep learning across 4 nodes with 4 GPUs each, which combination of directives correctly configures the job?

- **(a) [CORRECT]** #SBATCH --nodes=4 #SBATCH --ntasks-per-node=4 #SBATCH --gpus-per-node=4
- **(b)** #SBATCH --nodes=1 #SBATCH --ntasks=1 #SBATCH --gpus=16
- **(c)** #SBATCH --partition=all #SBATCH --threads=100
- **(d)** #SBATCH --nodes=16 #SBATCH --exclusive=no #SBATCH --time=unlimited

> **Correct Answer**: **(A)**
> **Bloom's Cognitive Level**: `Application`
> **Architectural Rationale**: 4 nodes (`--nodes=4`), 4 tasks per node (`--ntasks-per-node=4` , yielding 16 total MPI tasks), and 4 GPUs per node (`--gpus-per-node=4`) correctly assigns one process per accelerator across the 4 physical hosts.

Question 22

What is the consequence of failing to set '#SBATCH --cpus-per-task' appropriately in a multi-GPU deep learning Slurm batch script?

- **(a)** The job script will fail to compile Python code.
- **(b) [CORRECT]** Slurm defaults to allocating only 1 CPU core per GPU task, causing intense CPU data loading and preprocessing bottlenecks that starve the GPUs and cause massive idle stalls.
- **(c)** The GPUs will automatically run at half voltage.
- **(d)** The InfiniBand network adapter will disable RDMA.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Analyze`
> **Architectural Rationale**: By default, Slurm allocates 1 CPU core per task. If training requires data decoding and augmentation in `tf.data` , a single CPU core cannot keep up with high-speed GPU tensor ingest, throttling GPU utilization.

Question 23

What information does the Slurm command 'sacct -j --format=JobID,JobName,AllocCPUS,Elapsed,State,MaxRSS,ExitCode' provide to an HPC engineer?

- **(a)** A forecast of the electricity bill for the server.
- **(b) [CORRECT]** Detailed accounting records for the completed or failed job, including peak resident memory consumption (MaxRSS), total CPU count, runtime duration, exit status, and termination reason.
- **(c)** The source code of the Python script submitted.
- **(d)** The list of other users logged in via SSH.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Application`
> **Architectural Rationale**: `sacct` queries the Slurm accounting database (`slurmdbd`), returning post-mortem metrics such as peak RAM usage (`MaxRSS`), exit codes (e.g., 0:0 for success, 137 for OOM kills), and exact elapsed wall time.

Question 24

When a Slurm job fails with exit code 137 (SIGKILL) and state 'OUT_OF_MEMORY', what occurred during execution?

- **(a)** The GPU driver crashed due to cosmic ray bit flips.
- **(b) [CORRECT]** The Linux kernel Out-Of-Memory (OOM) killer or Slurm's cgroup memory enforcement subsystem terminated the job for exceeding its requested '#SBATCH --mem' allocation.
- **(c)** The user manually cancelled the job using 'scancel'.
- **(d)** The Slurm controller ran out of disk space on the head node.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Understand`
> **Architectural Rationale**: Exit code 137 corresponds to $128 + 9$ (SIGKILL). When a process exceeds the cgroup memory limit configured by `--mem`, the kernel or Slurm cgroup daemon sends SIGKILL immediately to protect node stability.

Question 25

In a distributed Slurm environment, why is it recommended to launch multi-node MPI/Horovod applications using 'srun' rather than standalone 'mpirun'?

- **(a)** Because mpirun cannot run C++ programs.
- **(b) [CORRECT]** Because 'srun' directly queries the Slurm job step manager, automatically inheriting node allocations, task counts, CPU affinity masks, and GRES GPU bindings without requiring manual hostfile creation.
- **(c)** Because srun bypasses Linux user permissions.
- **(d)** Because mpirun disables high-speed InfiniBand interfaces.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Analyze`
> **Architectural Rationale**: `srun` is natively integrated with Slurm. It automatically reads `SLURM_JOB_NODELIST`, establishes environment variables, sets up process cgroups, and manages output streaming without needing external SSH-based hostfiles.

Question 26

What does the Slurm directive '#SBATCH --gres=gpu:4' specify to the resource manager?

- **(a)** That the job requires 4 gigabytes of network buffer.
- **(b) [CORRECT]** That the job requests 4 Generic Resources (GRES) of type GPU on the allocated node.
- **(c)** That the job must run for exactly 4 minutes.
- **(d)** That 4 different users will share the job output.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Knowledge`
> **Architectural Rationale**: `--gres` (Generic Resources) instructs Slurm to allocate specific non-standard hardware devices—here, exactly 4 physical GPUs per node—and sets cgroup isolation so the job only sees those 4 devices.

Question 27

How does Slurm enforce CPU core pinning and memory locality across NUMA domains for distributed deep learning tasks?

- **(a) [CORRECT]** By using the '--cpu-bind=cores' and '--mem-bind=local' directives combined with Linux cgroups.

- **(b)** By rebooting the server into single-user mode.
- **(c)** By re-soldering the CPU sockets onto the motherboard.
- **(d)** By disabling the L3 cache in BIOS.

> **Correct Answer**: **(A)**
> **Bloom's Cognitive Level**: `Understand`
> **Architectural Rationale**: Using `--cpu-bind=cores` binds each task to specific physical CPU cores in its assigned NUMA socket. `--mem-bind=local` forces memory allocations to local DRAM channels, eliminating cross-socket UPI traffic.

Question 28

What does a pending job status with 'Reason: Priority' in 'squeue' indicate?

- **(a)** The job failed with a syntax error.
- **(b) [CORRECT]** The job is waiting in the queue because higher-priority jobs (determined by Multifactor Priority policies like fair-share, job age, and partition weight) are scheduled ahead of it.
- **(c)** The compute nodes are broken and undergoing repair.
- **(d)** The user forgot to provide a script filename.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Understand`
> **Architectural Rationale**: `Reason: Priority` means the cluster currently has or will have resources, but other queued jobs have higher calculated priority in the Slurm multifactor scheduling algorithm.

Question 29

Why is an HPC cluster administrator's 'Prolog' and 'Epilog' script configuration crucial for multi-user deep learning clusters?

- **(a)** To print welcome poetry when users log in.
- **(b) [CORRECT]** To sanitize the node before and after each job: verifying GPU health, flushing residual GPU memory, resetting PCIe clocks, and terminating orphaned user processes to prevent state contamination.
- **(c)** To convert markdown files to PDF automatically.
- **(d)** To backup user home directories to tape drives.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Analyze`
> **Architectural Rationale**: Epilog scripts execute with root permissions when a job terminates. They ensure lingering Python worker processes are killed, verify that GPU memory is completely freed, and reset device permissions for the next job.

Question 30

In a Slurm heterogeneous job or array job, what does the environment variable '$SLURM_ARRAY_TASK_ID' represent?

- **(a)** The total number of tasks in the entire cluster.

- **(b) [CORRECT]** The specific unique index of the current sub-task within a Slurm job array, allowing one batch script to process different hyperparameter combinations or data shards.
- **(c)** The process ID of the Slurm daemon.
- **(d)** The version of Python installed on the node.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Application`
> **Architectural Rationale**: When launching array jobs (`#SBATCH --array=0-99`), Slurm submits 100 identical job steps. Each instance receives its unique index in `$SLURM_ARRAY_TASK_ID` to select different hyperparameters, cross-validation folds, or input files.

Question 31

In the Ring-AllReduce algorithm, how many logical phases are executed, and what are their names?

- **(a)** 1 phase: Direct Broadcast.
- **(b) [CORRECT]** 2 phases: Scatter-Reduce (where each worker accumulates a slice of the final sum) followed by AllGather (where the accumulated slices are distributed to all workers).
- **(c)** 3 phases: Map, Shuffle, and Reduce.
- **(d)** 4 phases: Encrypt, Compress, Send, and Decrypt.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Understand`
> **Architectural Rationale**: Ring-AllReduce splits tensors of size M into N chunks. Phase 1 (Scatter-Reduce) runs for $(N-1)$ steps where chunks are passed along a logical ring and accumulated. Phase 2 (AllGather) runs for another $(N-1)$ steps to circulate the finalized values to all N workers.

Question 32

Why is the communication time in Ring-AllReduce described as 'bandwidth-optimal' and independent of the number of worker nodes N?

- **(a)** Because data is transmitted through quantum entanglement.
- **(b) [CORRECT]** Because the total data transferred by each worker per step is $2 \times \frac{N-1}{N} \times M$, which asymptotically approaches a constant $2M$ as N becomes large, meaning network bandwidth per node does not degrade as the cluster scales.
- **(c)** Because all workers share a single physical hard drive.
- **(d)** Because gradients are never transferred across the network.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Analyze`
> **Architectural Rationale**: Unlike Parameter Server architectures where master network bandwidth scales as $O(N)$, every worker in Ring-AllReduce only sends and receives $2 \times \frac{N-1}{N} \times M$ bytes. For large N, this is $\approx 2M$, remaining completely constant regardless of cluster size.

Question 33

In Horovod, why must dataset sharding be implemented via 'dataset.shard(num_shards=hvd.size(), index=hvd.rank())'?

- **(a)** To compress image files into zip archives.
- **(b) [CORRECT]** To ensure that each distributed worker processes a mutually disjoint, unique partition of the training dataset, avoiding redundant duplicate gradient computation.
- **(c)** To convert images into audio files.
- **(d)** To encrypt the training data across workers.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Apply`
> **Architectural Rationale**: Without sharding, every worker would process the exact same training samples in the same order, defeating the purpose of Data Parallelism. `dataset.shard` partitions the dataset so each rank receives an exclusive $1/N$-th slice.

Question 34

In a distributed training script using Horovod with TensorFlow, why is the checkpoint saving code wrapped in 'if hvd.rank() == 0:'?

- **(a)** Because worker ranks other than 0 do not have access to model weights.
- **(b) [CORRECT]** To prevent all N concurrent workers from simultaneously writing to the exact same checkpoint file path on the shared filesystem, which would cause severe file lock corruption and I/O thrashing.
- **(c)** Because rank 0 is the only process with a Python interpreter.
- **(d)** To delete model weights after training finishes.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Understand`
> **Architectural Rationale**: Because all workers maintain synchronized weights after every step, writing the model checkpoint from multiple ranks simultaneously onto a shared parallel filesystem (NFS/Lustre) creates write collisions, file corruption, and massive I/O serialization.

Question 35

What is 'Tensor Fusion' in Horovod, controlled by the environment variable 'HOROVOD_FUSION_THRESHOLD'?

- **(a)** Combining two physical GPUs into one single GPU die.
- **(b) [CORRECT]** A performance feature that packs multiple small gradient tensors into a single contiguous fusion buffer before calling Ring-AllReduce, amortizing network transfer latency.
- **(c)** A technique to fuse text and image models together.
- **(d)** A compiler pass that deletes unused convolutional filters.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Analyze`
> **Architectural Rationale**: Calling AllReduce on thousands of tiny gradient tensors incurs high latency overhead per call. Tensor Fusion groups small tensors into a buffer (e.g., 64 MB default) so that fewer, larger AllReduce operations are dispatched, saturating link bandwidth.

Question 36

What occurs if a developer forgets to call 'hvd.broadcast_variables(model.variables, root_rank=0)' at the start of Horovod training?

- **(a)** The script will immediately raise a SyntaxError.
- **(b) [CORRECT]** Each worker starts with different randomly initialized weights; averaging gradients computed from divergent parameter spaces causes the model to fail to converge and produce garbage predictions.
- **(c)** The computer will reboot automatically.
- **(d)** The dataset will be deleted from the filesystem.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Analyze`
> **Architectural Rationale**: In Data Parallelism, mathematical convergence requires that all replicas evaluate gradients on identical parameters W_t. If initial weights W_0 differ across ranks, gradient updates point in conflicting directions, completely destroying model convergence.

Question 37

How does Horovod's 'Auto-Tuning' feature (`--autotune`) optimize distributed training performance during runtime?

- **(a)** By automatically rewriting the neural network architecture in PyTorch.
- **(b) [CORRECT]** By dynamically searching for the optimal values of fusion threshold, cycle time, and response cache size on the active cluster fabric during early training epochs.
- **(c)** By increasing the GPU fan speed to 100%.
- **(d)** By downloading pre-trained weights from Hugging Face.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Understand`
> **Architectural Rationale**: Horovod autotuning runs a Bayesian optimization / dynamic search over internal parameters (such as `HOROVOD_FUSION_THRESHOLD` and `HOROVOD_CYCLE_TIME`) to find the highest-throughput configuration for the specific network topology.

Question 38

When scaling from 1 worker to 64 workers with Horovod, why is 'Gradual Learning Rate Warmup' recommended alongside the Linear Scaling Rule?

- **(a)** To allow the GPU silicon to warm up to operating temperature.
- **(b) [CORRECT]** Because multiplying the learning rate by 64 immediately at step 0 causes severe gradient instability and divergence, as initial random weights produce large, volatile gradients.
- **(c)** To ensure that all worker nodes connect to the internet.
- **(d)** Because Slurm kills jobs that use high learning rates early on.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Understand`
> **Architectural Rationale**: In early epochs, random weights yield high gradient variance. Applying a massive learning rate ($64 \times LR$) immediately leads to exploding gradients and divergent loss. Gradual warmup ramps the learning rate linearly from base to target over 5 epochs.

Question 39

In Horovod, what is the role of the 'Response Cache' feature?

- **(a)** Storing web page responses in a local Redis cache.
- **(b) [CORRECT]** Caching tensor metadata and collective negotiation decisions between ranks across iterations to reduce coordinator rank CPU overhead.
- **(c)** Caching JPEG images in uncompressed bitmap format.
- **(d)** Storing student test scores in an SQLite table.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Knowledge`
> **Architectural Rationale**: Horovod workers must coordinate which tensors are ready for reduction. The Response Cache remembers tensor negotiation outcomes across steps, eliminating redundant coordinator communication for static execution graphs.

Question 40

Which command correctly launches a Horovod training script across 4 nodes with 4 GPUs each (16 total processes) using the 'horovodrun' wrapper?

- **(a) [CORRECT]** horovodrun -np 16 -H server1:4,server2:4,server3:4,server4:4 python train.py
- **(b)** python train.py --horovod --nodes=4
- **(c)** srun -n 1 python train.py
- **(d)** mpirun -np 1 train.py

> **Correct Answer**: **(A)**
> **Bloom's Cognitive Level**: `Application`
> **Architectural Rationale**: `horovodrun -np 16 -H host1:slots,host2:slots...` specifies the total process count (16) and maps 4 task slots to each of the 4 cluster hosts, launching the distributed training ring cleanly.

Question 41

What is the primary difference between 'Weak Scaling' and 'Strong Scaling' when benchmarking distributed deep learning clusters?

- **(a)** Weak scaling uses low-cost GPUs; strong scaling uses high-end accelerators.
- **(b) [CORRECT]** In Strong Scaling, the total global batch size is kept constant as nodes increase (testing time-to-solution reduction); in Weak Scaling, the batch size per node is kept constant as nodes increase (total problem size grows proportionally with compute).
- **(c)** Strong scaling applies only to convolutional networks; weak scaling applies only to Transformers.

- **(d)** Weak scaling does not use network synchronization.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Analyze`
> **Architectural Rationale**: Strong Scaling keeps total work fixed while adding processors (evaluating speedup on fixed problem size). Weak Scaling scales the problem size proportionally with processor count (evaluating throughput on expanding problem sizes).

Question 42

In distributed deep learning, why does Strong Scaling inevitably hit an upper scaling limit where adding more nodes yields zero or negative speedup?

- **(a)** Because Linux cannot support more than 16 IP addresses.
- **(b) [CORRECT]** As N increases, the local batch size per worker (B/N) shrinks, causing local arithmetic intensity to drop into memory-bandwidth bound territory while communication latency overhead begins to dominate step time.
- **(c)** Because neural networks forget their weights when distributed across nodes.
- **(d)** Because Python limits floating point operations to 1 billion per day.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Analyze`
> **Architectural Rationale**: When batch per worker becomes too small (e.g., 2 images per GPU), GPUs spend minimal time on matrix compute and excessive time waiting on AllReduce synchronization and PCIe latency.

Question 43

How does 'Communication-Computation Overlap' in modern distributed frameworks (e.g., Horovod, PyTorch DDP) boost scaling efficiency?

- **(a)** By running data loading in the background while the computer is turned off.
- **(b) [CORRECT]** By triggering the AllReduce gradient synchronization of final layers immediately in the background while earlier layers are still actively executing their backward pass computations.
- **(c)** By skipping the backward pass entirely on even-numbered epochs.
- **(d)** By converting all 32-bit floats into 1-bit booleans.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Understand`
> **Architectural Rationale**: In backpropagation, gradients for outer layers (e.g., layer 100) are ready long before inner layers (e.g., layer 1). Overlapping begins transmitting layer 100 gradients over the network while layer 99-1 are still being computed on the GPU, hiding communication latency.

Question 44

If an HPC cluster monitoring profile indicates that GPU compute utilization is only 18% during distributed training, what is the MOST probable root cause?

- **(a)** The GPU clock frequency has been locked by the vendor.

- **(b) [CORRECT]** The input data pipeline (CPU data loading, JPEG decompression, augmentation, or slow storage I/O) is starved, causing the GPUs to spend over 80% of their time stalled waiting for incoming batches.
- **(c)** The model has converged to zero training loss.
- **(d)** The learning rate is too large.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Analyze`
> **Architectural Rationale**: Low GPU compute utilization during training almost universally stems from I/O starvation: the CPU preprocessing pipeline (`tf.data`) cannot feed batches over the PCIe bus fast enough to saturate accelerator compute cores.

Question 45

What is the purpose of 'Gradient Clipping by Global Norm' (`tf.clip_by_global_norm`) during large-scale distributed training?

- **(a)** To delete negative numbers from the dataset.
- **(b) [CORRECT]** To prevent exploding gradients from causing numerical overflow (NaN/Inf) and parameter corruption when large effective batch sizes produce large gradient spikes.
- **(c)** To crop images to a square aspect ratio.
- **(d)** To limit the maximum number of training epochs to 100.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Understand`
> **Architectural Rationale**: Gradient clipping computes the global L_2 norm of all gradients across all layers. If the norm exceeds a threshold c, gradients are scaled down proportionally ($g \leftarrow g \times \frac{c}{\|g\|}$), preventing parameter explosion.

Question 46

In high-performance RoCE v2 networks, why is Priority Flow Control (PFC) configured on enterprise Ethernet switches?

- **(a)** To allow video streaming to take priority over scientific data.
- **(b) [CORRECT]** To eliminate packet drops by pausing specific traffic classes (IEEE 802.1Qbb) when switch buffer thresholds are approached, fulfilling RDMA's requirement for a lossless link layer.
- **(c)** To convert IP addresses into domain names automatically.
- **(d)** To encrypt Ethernet frame payloads.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Understand`
> **Architectural Rationale**: RoCE v2 requires a lossless Ethernet fabric because packet drops trigger expensive retransmissions that destroy RDMA performance. PFC issues PAUSE frames on dedicated priority queues to prevent buffer overflow without dropping packets.

Question 47

What does a sudden loss value of 'NaN' (Not-a-Number) during distributed mixed-precision training typically indicate, and how is it resolved?

- **(a)** The hard drive is full; resolved by deleting log files.
- **(b) [CORRECT]** Numerical overflow in 16-bit floating point representations (values exceeding 65,504 in FP16); resolved by enabling dynamic loss scaling or transitioning to BF16 (which supports dynamic range up to 10^{38}).
- **(c)** The network cable has become unplugged; resolved by rebooting switches.
- **(d)** The model has achieved 100% test accuracy.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Analyze`
> **Architectural Rationale**: FP16 caps at 65,504. Large activations or gradients overflow to ∞, which yields `NaN` when subtracted or normalized. Dynamic loss scaling scales down values, or switching to BF16 (which matches FP32's 8-bit exponent) completely prevents overflow.

Question 48

Why is storing deep learning datasets in container formats like TFRecord, HDF5, or WebDataset superior to storing millions of individual raw image files on parallel filesystems like Lustre?

- **(a)** Because TFRecords make images look sharper.
- **(b) [CORRECT]** Because packaging millions of files into sequential binary chunks converts random small metadata requests into high-throughput contiguous block streaming I/O, saturating Lustre Object Storage Targets (OSTs).
- **(c)** Because Linux cannot open more than 1000 files in a single directory.
- **(d)** Because container formats automatically train the model without Python.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Analyze`
> **Architectural Rationale**: Parallel file systems excel at high-bandwidth sequential block I/O. Bundling small files into large contiguous binary records prevents metadata server (MDS) lock contention and achieves multi-gigabyte/sec storage throughput.

Question 49

How does 'Pipeline Parallelism' (e.g., GPipe) overcome the memory barrier of models that cannot fit on a single accelerator?

- **(a)** By storing all model weights on an external network hard drive.
- **(b) [CORRECT]** By partitioning the sequential layers of a deep network across multiple accelerators and dividing the minibatch into smaller microbatches to pipeline execution across stages.
- **(c)** By converting all model layers into linear equations.
- **(d)** By running training only on the CPU host memory.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Understand`
> **Architectural Rationale**: Pipeline parallelism assigns consecutive layers to a chain of accelerators. To prevent idle time (the pipeline bubble), minibatches are divided into microbatches, allowing downstream accelerators to begin computing while upstream ones process subsequent microbatches.

Question 50

In enterprise HPC clusters, what is the role of Intel oneAPI Collective Communications Library (oneCCL) in multi-node deep learning?

- **(a)** It manages user email subscriptions.
- **(b) [CORRECT]** It provides cross-architecture, hardware-accelerated collective communication primitives (Allreduce, Allgather, Reduce-scatter) optimized for Intel Xeon CPUs, Xe GPUs, and high-speed fabrics.
- **(c)** It is an interactive drawing tool for convolutional neural networks.
- **(d)** It replaces standard Linux bash commands with PowerShell.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Knowledge`
> **Architectural Rationale**: oneCCL is Intel's oneAPI collective communications library. It is designed specifically for scaling deep learning workloads across heterogeneous Intel compute architectures (CPUs and Xe XPUs) over high-performance fabrics.
